

TP 1

GPIO et mesures de performances

L'objectif de ce premier TP est de comparer une approche très bas niveau via les registres avec une approche plus haut niveau via une librairie. Pour cela, nous allons utiliser une carte Arduino Uno R3.

Documentation carte : <https://docs.arduino.cc/hardware/uno-rev3/>

Documentation librairie : <https://docs.arduino.cc/language-reference/>

Documentation microcontrôleur : https://chamilo.grenoble-inp.fr/courses/ESISAR4AMSN420/document/Atmel_doc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

Documentation assembleur : https://chamilo.grenoble-inp.fr/courses/ESISAR4AMSN420/document/Atmel_doc/atmel-0856-avr-instruction-set-manual.pdf

Documentation compilateur : https://avrdudes.github.io/avr-libc/avr-libc-user-manual/group__avr__interrupts.html

Pour toutes les questions/recherches dans la documentation, vous indiquerez le document, la section (par exemple 1.2.3) et numéro de page où vous avez trouvé l'information.

1.1 Préparation individuelle

Pour avoir plus de temps pour l'expérimentation pendant la séance, vous devez préparer la séance en répondant aux questions suivantes :

- À partir de la documentation de la carte, identifier la référence du microcontrôleur ainsi que le port et le pin sur lequel est branché la LED.
- Quelle est la différence entre le port PB5 et le port D13?
- À partir de la documentation du microcontrôleur (chapitre 13), identifier les registres à configurer et les valeurs associées pour avoir le pin de la LED en sortie haute.
- À partir de la documentation assembleur et de la documentation microcontrôleur, comment utiliser l'instruction SBI pour inverser le port PB5? Donner la ligne assembleur correspondante.
- À partir de la documentation de la carte, identifier la référence de l'oscillateur cristal.
- À partir de la documentation de l'oscillateur (à chercher vous même), identifier ce à quoi chacun des champs de la référence correspond. Quelle est la précision du cristal à 25 °C au bout de 1 an?

1.2 GPIO

Nous allons comparer la différence de performance entre 4 méthodes pour faire clignoter une LED : deux méthodes en C avec les registres matériels, une en assembleur avec les registres également et la dernière en C avec une librairie plus haut-niveau (plus abstraite).

- Faire la configuration des registres pour obtenir le mode Push-Pull et allumer la LED. Pour cela, il faut inclure l'en-tête spécifique à notre Arduino UNO R3 pour avoir accès aux registres. Le compilateur `avr-gcc` fournit une interface unique `#include <avr/io.h>` qui lui même inclut le bon en-tête de notre cible (ici `avr/iom328p.h`)
- Dans une boucle, faire clignoter la LED en utilisant le registre d'écriture et mesurer la fréquence à l'oscilloscope.
- De même en utilisant cette fois le registre permettant d'inverser l'état du pin (*toggle*).
- Laquelle des 2 méthodes est la plus rapide? Pourquoi? Vous pouvez regarder le code assembleur de votre binaire en utilisant l'utilitaire `avr-objdump`. L'IDE Arduino génère le binaire compilé en faisant  `Export compiled binary` dans le dossier de votre projet Arduino :
`avr-objdump -DS yourprogram.ino.elf > yourprogram.txt`
Le drapeau `-D` indique de désassembler toutes les sections (y compris les données) et le drapeau `-S` entremêle l'assembleur généré avec votre code source.
- De même en utilisant les instructions `sbi` et `rjmp` dans un bout de code assembleur *inline*. La syntaxe est :

```
asm volatile("ligne asm1\n"  
            "ligne asm2\n")
```

```
"ligne asm3\n");
```

- Faire clignoter la LED en utilisant la librairie Arduino et la fonction `digitalWrite` et mesurer la fréquence. En sachant que le microcontrôleur fonctionne à 16 MHz, combien de cycles d'horloge sont nécessaires pour un simple *toggle* ? Comparer avec les solutions précédentes.
- Faire clignoter la LED en utilisant la librairie Arduino et une attente logicielle pour obtenir une fréquence de 1 kHz. De manière aléatoire (par exemple 1 fois sur 2), ajouter une sortie sur le port série réglé à 9 600 bauds d'un message quelconque (au moins 10 caractères). Quel est l'effet sur la fréquence de clignotement de la LED ?
- Même chose que précédemment mais sans attente logicielle.
- Comparer les 2 codes précédents. Quels sont les avantages et inconvénients de ces 2 codes ?

1.3 Préparation individuelle

- À partir de la documentation de la librairie de la fonction `attachInterrupt`, quels pins sont disponibles pour des interruptions ? Identifier les pins correspondants du microcontrôleur.
- À partir de la documentation du microcontrôleur, quelles sont les interruptions disponibles sur les pins précédemment identifiés ? Sur quel pin se trouve l'interruption externe 0 ?
Indice : regarder les fonctions alternatives des GPIO
- Quelle est la différence entre les interruptions `INT0` et `PCINT0` ?
- À partir du chapitre sur les interruptions externes dans la documentation microcontrôleur, identifier les registres et les valeurs requises pour configurer l'interruption externe 0 (`INT0`) sur front montant.
- À partir de la documentation du compilateur `avr-gcc`, identifier les noms des vecteurs d'interruption pour les interruptions `INT0` et `PCINT2`.

1.4 Interruption

Nous allons maintenant comparer les performances sur les interruptions en comparant la librairie avec une approche bas niveau. Pour cela, nous allons utiliser l'interruption externe 0 et brancher un GBF dessus avec un signal carré à 1 kHz avec une amplitude de 3.3 V et un offset de 1.65 V. Régler également la sortie du GBF en mode haute impédance. Vérifier que votre maquette est bien également en adaptation haute-impédance !

- Configurer l'interruption externe 0 sur front montant.
- Écrire la fonction d'interruption telle que : `ISR([vector_name]) { ... }`, en remplaçant `[vector_name]` par le nom identifié dans la préparation, et allumer la LED dans cette fonction. Éteindre la LED depuis la fonction `loop`.
- Mesurer le temps de réponse à l'oscilloscope en faisant l'écriture du registre et en passant par la fonction `digitalWrite`.
- Commenter votre code précédent et faire l'interruption en utilisant la fonction `attachInterrupt`.
- Mesurer le temps de réponse à l'oscilloscope. Conclure.

1.5 Timers

Nous allons désormais faire clignoter la LED en utilisant les timers de l'Arduino. Le timer 0 est déjà utilisé par la librairie Arduino, nous allons donc utiliser le timer 1 et obtenir une base de temps de 1 seconde.

- Quelle est la taille du timer 1 ? Quelle est la division d'horloge maximale du prescaler ? En déduire la période maximale du timer 1.
- En utilisant le mode normal du timer 1, quelle doit-être la division d'horloge pour être le plus proche d'une base de temps de 1 seconde ? Implémenter le mode normal et mesurer la fréquence à l'oscilloscope. La fonction d'interruption se déclare comme suit : `ISR(TIMER1_OVF_vect) { ... }`
- Proposer un réglage utilisant le mode CTC (mode 4 dans la table 15-5). Quelle division d'horloge et quelle valeur maximale pour le timer pour être le plus proche de 1 seconde ? Quelle registre contient la valeur maximale ? Implémenter cette solution et mesurer la fréquence à l'oscilloscope. L'interruption pour le mode CTC se déclare comme suit : `ISR(TIMER1_COMPA_vect) { ... }`
- Faire de même sans interruption et en utilisant l'un des modes PWM.